Notes on
LBM C++ Program

David Puerta

Contents

1 Overview

The program is an implementation of the Lattice Boltzmann Method in 2D. It is currently for Newtonian fluids in a rigid vessel but will be extended to non-Newtonian fluids in flexible vessels.

The program consists of 5 key steps and one decision step: initialisation of the simulation, the collision step, the streaming step, applying the boundary conditions, finding the macroscopic moments and finally the L_2-norm error calculation.

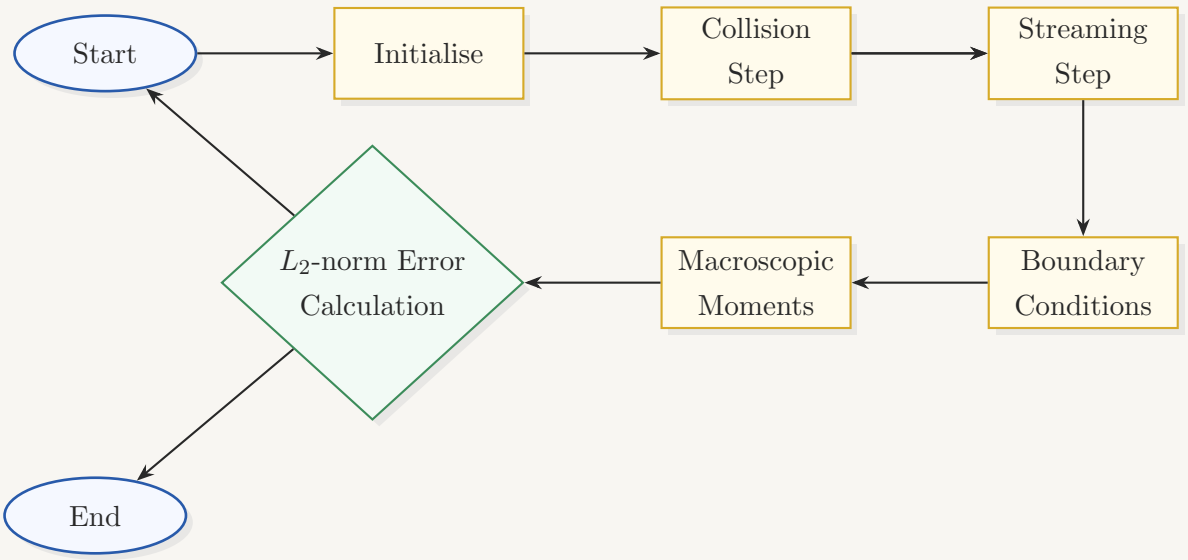


Figure 1: Overview of the LBM program flow.

2 Collision Step

After initialisation, the first thing the program does is the collision step of the LBM.

There are two types of collision steps: collision with a forcing term or without a forcing term. Currently implemented is the collision without a forcing term.

2.1 Collision Without a Forcing Term

For a collision step with no forcing term, the population $f_i(\mathbf{x}, t)$ is updated to $f_i^\star(\mathbf{x}, t)$, the population after a collision using,

$$f_i^\star(\mathbf{x}, t) = f_i(\mathbf{x}, t)\left(1 - \frac{\Delta t}{\tau}\right) + f_i^{\text{eq}}(\mathbf{x}, t)\frac{\Delta t}{\tau} \tag{3.9}$$

which is equation (3.9) from the LBM book by Kruger et al. This is the relaxation equation using the BGK operator where τ is the relaxation time and $f_i^{\text{eq}}(\mathbf{x}, t)$ is the equilibrium distribution. Using lattice units, equation (3.9) reads

$$f_i^\star(\mathbf{x}, t) = f_i(\mathbf{x}, t)\left(1 - \frac{1}{\tau}\right) + f_i^{\text{eq}}(\mathbf{x}, t)\frac{1}{\tau} \tag{C1}$$

which we will use to implement the collision step.

In order to find the equilibrium function, we use equation (3.4) from the LBM book by Kruger et al,

$$f_i^{\text{eq}}(\mathbf{x}, t) = w_i\rho\left(1 + \frac{\mathbf{u}\cdot\mathbf{c}_i}{c_s^2} + \frac{(\mathbf{u}\cdot\mathbf{c}_i)^2}{2c_s^4} - \frac{\mathbf{u}\cdot\mathbf{u}}{2c_s^2}\right) \tag{3.4}$$

where $\{\mathbf{c}_i, w_i\}$ is our velocity set and c_s is the speed of sound. Using lattice units in the D_2Q_9 lattice, we have $c_s^2 = 1/3$ and the values for our velocity set as

Direction i	0	1	2	3	4	5	6	7	8
Velocity $\mathbf{c}_i$	$(0,0)$	$(1,0)$	$(-1,0)$	$(0,1)$	$(0,-1)$	$(1,1)$	$(-1,1)$	$(-1,-1)$	$(1,-1)$
Weight w_i	4/9	1/9	1/9	1/9	1/9	1/36	1/36	1/36	1/36

Table 1: Velocity set for D_2Q_9

which we obtained from Table 3.1 in the LBM book by Kruger et al.

Finally, expanding the dot products in (3.4) and using our values for $\mathbf{c}_i$, w_i and c_s^2 leaves us with an expression for each of the 9 components of the equilibrium distribution,

Direction i	Equilibrium distribution $f_i^{\text{eq}}(\mathbf{x}, t)$
0	$w_0\rho\left[1 - \frac{3}{2}(\mathbf{u}\cdot\mathbf{u})\right]$
1	$w_1\rho\left[1 + 3u_x + \frac{9}{2}u_x^2 - \frac{3}{2}(\mathbf{u}\cdot\mathbf{u})\right]$
2	$w_2\rho\left[1 - 3u_x + \frac{9}{2}u_x^2 - \frac{3}{2}(\mathbf{u}\cdot\mathbf{u})\right]$
3	$w_3\rho\left[1 + 3u_y + \frac{9}{2}u_y^2 - \frac{3}{2}(\mathbf{u}\cdot\mathbf{u})\right]$
4	$w_4\rho\left[1 - 3u_y + \frac{9}{2}u_y^2 - \frac{3}{2}(\mathbf{u}\cdot\mathbf{u})\right]$
5	$w_5\rho\left[1 + 3(u_x + u_y) + \frac{9}{2}(u_x + u_y)^2 - \frac{3}{2}(\mathbf{u}\cdot\mathbf{u})\right]$
6	$w_6\rho\left[1 + 3(-u_x + u_y) + \frac{9}{2}(-u_x + u_y)^2 - \frac{3}{2}(\mathbf{u}\cdot\mathbf{u})\right]$
7	$w_7\rho\left[1 - 3(u_x + u_y) + \frac{9}{2}(-u_x - u_y)^2 - \frac{3}{2}(\mathbf{u}\cdot\mathbf{u})\right]$
8	$w_8\rho\left[1 + 3(u_x - u_y) + \frac{9}{2}(u_x - u_y)^2 - \frac{3}{2}(\mathbf{u}\cdot\mathbf{u})\right]$

Table 2: Expression for $f_i^{\text{eq}}(\mathbf{x}, t)$

which we will use in the implementation.

2.2 Implementation of Collision Step Without a Forcing Term

Currently in the main file of the program, we have the collision step with no forcing term. For this, we call the collision function which itself calls the equilibrium function and then preforms the collision step calculation.

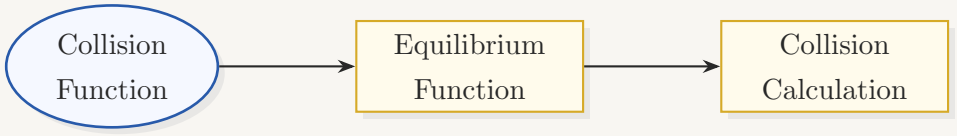


Figure 2: Flow of the Collision No Force implementation.

Inside a for loop, the equilibrium function is called to get the equilibrium function $f_i^{\text{eq}}(\mathbf{x}, t)$, which is now called feq.

2.2.1 Equilibrium Function

The function calculates the value of the equilibrium function feq using the expressions in Table 2.

```cpp
void equilibrium(const int k, double feq[9])
{

    double ux2 = ux[k] * ux[k];
    double uy2 = uy[k] * uy[k];
    double term1 = 1.0 - 1.5 * (ux2 + uy2);
    double term2 = 4.5 * (ux[k] + uy[k]) * (ux[k] + uy[k]);
    double term3 = 4.5 * (ux[k] - uy[k]) * (ux[k] - uy[k]);

    feq[0] = w[0] * rho[k] * term1;
    feq[1] = w[1] * rho[k] * (term1 + 3.0 * ux[k] + 4.5 * ux2);
    feq[2] = w[2] * rho[k] * (term1 + 3.0 * uy[k] + 4.5 * uy2);
    feq[3] = w[3] * rho[k] * (term1 - 3.0 * ux[k] + 4.5 * ux2);
    feq[4] = w[4] * rho[k] * (term1 - 3.0 * uy[k] + 4.5 * uy2);
    feq[5] = w[5] * rho[k] * (term1 + term2 + 3.0 * (ux[k] + uy[k]));
    feq[6] = w[6] * rho[k] * (term1 + term3 - 3.0 * (ux[k] - uy[k]));
    feq[7] = w[7] * rho[k] * (term1 + term2 - 3.0 * (ux[k] + uy[k]));
    feq[8] = w[8] * rho[k] * (term1 + term3 + 3.0 * (ux[k] - uy[k]));

}

```

Figure 3: (equilibrium_function.cpp) Equilibrium function

2.2.2 Collision Calculation

Using feq, it preforms the collision step calculation in (C1) to find the updated distribution after the collision $f_i^\star(\mathbf{x}, t)$, which is now called $fcol$.

```cpp
void collision()
{

    for (int k = 0; k < N; ++k)
    {

        double feq[Nvel];

        equilibrium(k, feq);

        // Since f_0 is a rest population, i.e. its value does not change
        // during the streaming step, the collision step is directly assigned
        // to the variable f0, thus saving memory by not creating another variable fcol0.
        f0[k] = omomega * f0[k] + omega * feq[0];

        fcol1[k] = omomega * f1[k] + omega * feq[1];
        fcol2[k] = omomega * f2[k] + omega * feq[2];
        fcol3[k] = omomega * f3[k] + omega * feq[3];
        fcol4[k] = omomega * f4[k] + omega * feq[4];
        fcol5[k] = omomega * f5[k] + omega * feq[5];
        fcol6[k] = omomega * f6[k] + omega * feq[6];
        fcol7[k] = omomega * f7[k] + omega * feq[7];
        fcol8[k] = omomega * f8[k] + omega * feq[8];

    }

}
```

Figure 4: (collision_step.cpp) Collision function

2.3 Collision With a Forcing Term

Similar to collision without a forcing term, for a collision step with a forcing term, the population $f_i(\mathbf{x}, t)$ is updated to $f_i^\star(\mathbf{x}, t)$, the population after a collision using,

$$f_i^\star(\mathbf{x}, t) = f_i(\mathbf{x}, t)\left(1 - \frac{1}{\tau}\right) + f_i^{\text{eq}}(\mathbf{x}, t)\frac{1}{\tau} + \Omega_i^F(\mathbf{x}, t) \tag{C2}$$

which is a modified version of equation (13) from LBM review by Dzanic et al, where we used the BGK operator for $\Omega_i^{SRT}(\mathbf{x}, t)$, lattice units and re-indexed with i.

The additional forcing term $\Omega_i^F(\mathbf{x}, t)$ is calculated using the Guo et al method,

$$\Omega_i^F(\mathbf{x}, t) = \left(1 - \frac{\Delta t}{\tau}\right) w_i \left(\frac{\mathbf{c}_i - \mathbf{u}}{c_s^2} + \frac{(\mathbf{c}_i \cdot \mathbf{u})\mathbf{c}_i}{c_s^4}\right) \cdot \mathbf{F} \tag{6.1.1}$$

which is found on the first row of Table 6.1 in the LBM book by Kruger et al. Using lattice units in the D_2Q_9 lattice, and the previous values for our velocity set $\{\mathbf{c}_i, w_i\}$, we can expand the dot products in (6.1.1) which leaves us with an expression for each of the 9 components of the forcing term,

Direction i	Force term Ω_i^F
0	$\left(1 - \frac{1}{2}\omega\right) w_0 \left[3(-u_x F_x - u_y F_y)\right]$
1	$\left(1 - \frac{1}{2}\omega\right) w_1 \left[3(F_x - u_x F_x - u_y F_y) + 9u_x F_x\right]$
2	$\left(1 - \frac{1}{2}\omega\right) w_2 \left[3(-F_x - u_x F_x - u_y F_y) + 9u_x F_x\right]$
3	$\left(1 - \frac{1}{2}\omega\right) w_3 \left[3(-u_x F_x + F_y - u_y F_y) + 9u_y F_y\right]$
4	$\left(1 - \frac{1}{2}\omega\right) w_4 \left[3(-u_x F_x - F_y - u_y F_y) + 9u_y F_y\right]$
5	$\left(1 - \frac{1}{2}\omega\right) w_5 \left[3(F_x - u_x F_x + F_y - u_y F_y) + 9(u_x + u_y)(F_x + F_y)\right]$
6	$\left(1 - \frac{1}{2}\omega\right) w_6 \left[3(-F_x - u_x F_x + F_y - u_y F_y) + 9(-u_x + u_y)(-F_x + F_y)\right]$
7	$\left(1 - \frac{1}{2}\omega\right) w_7 \left[3(-F_x - u_x F_x - F_y - u_y F_y) + 9(u_x + u_y)(F_x + F_y)\right]$
8	$\left(1 - \frac{1}{2}\omega\right) w_8 \left[3(F_x - u_x F_x - F_y - u_y F_y) + 9(u_x - u_y)(F_x - F_y)\right]$

Table 3: Expression for the forcing term Ω_i^F

which we will use in the implementation.

2.4 Implementation of Collision Step With a Forcing Term

For this, we call the collision with force function, which itself calls the equilibrium function Guo forcing function and then preforms the collision step calculation.

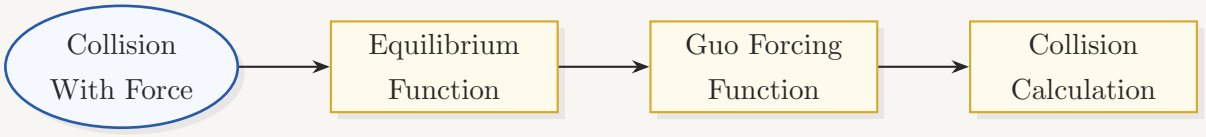


Figure 5: Flow of the Collision With Force implementation.

Inside a for loop, the equilibrium function is called to get the equilibrium function $f_i^{\text{eq}}(\mathbf{x}, t)$, which is now called feq.

2.4.1 Guo Forcing Function

This function calculates the forcing function $\Omega_i^F(\mathbf{x}, t)$, which is now called $Flattice$, using the expressions in Table 3.

```
70  void Guo_forcing(const int k, double Flattice[9])
71  {
72
73      double term1 = - 3.0 * (ux[k] * fx[k] + uy[k] * fy[k]);
74      double term2 = 3.0 * (ux[k] + uy[k]);
75      double term3 = 3.0 * (fx[k] + fy[k]);
76      double term4 = 3.0 * (ux[k] - uy[k]);
77      double term5 = 3.0 * (fx[k] - fy[k]);
78
79      Flattice[0] = omhomega * w[0] * term1;
80      Flattice[1] = omhomega * w[1] * (term1 + 3.0 * fx[k] + 9.0 * ux[k] * fx[k]);
81      Flattice[2] = omhomega * w[2] * (term1 + 3.0 * fy[k] + 9.0 * uy[k] * fy[k]);
82      Flattice[3] = omhomega * w[3] * (term1 - 3.0 * fx[k] + 9.0 * ux[k] * fx[k]);
83      Flattice[4] = omhomega * w[4] * (term1 - 3.0 * fy[k] + 9.0 * uy[k] * fy[k]);
84      Flattice[5] = omhomega * w[5] * (term1 + term3 + term2 * term3);
85      Flattice[6] = omhomega * w[6] * (term1 - term5 + term4 * term5);
86      Flattice[7] = omhomega * w[7] * (term1 - term3 + term2 * term3);
87      Flattice[8] = omhomega * w[8] * (term1 + term5 + term4 * term5);
88
89  }
90
```

Figure 6: (collision_step.cpp) Guo forcing function

2.4.2 Collision Calculation with Force

Using feq and $Flattice$, it preforms the collision step calculation in (C2) to find the updated distribution after the collision $f_i^\star(\mathbf{x}, t)$, which is now called $fcol$.

```
41  void collision_with_force()
42  {
43
44      for (int k = 0; k < N; ++k)
45      {
46
47          double feq[Nvel];
48
49          equilibrium(k, feq);
50
51          double Flattice[Nvel];
52
53          Guo_forcing(k, Flattice);
54
55          f0[k] = omomega * f0[k] + omega * feq[0] + Flattice[0];
56
57          fcol1[k] = omomega * f1[k] + omega * feq[1] + Flattice[1];
58          fcol2[k] = omomega * f2[k] + omega * feq[2] + Flattice[2];
59          fcol3[k] = omomega * f3[k] + omega * feq[3] + Flattice[3];
60          fcol4[k] = omomega * f4[k] + omega * feq[4] + Flattice[4];
61          fcol5[k] = omomega * f5[k] + omega * feq[5] + Flattice[5];
62          fcol6[k] = omomega * f6[k] + omega * feq[6] + Flattice[6];
63          fcol7[k] = omomega * f7[k] + omega * feq[7] + Flattice[7];
64          fcol8[k] = omomega * f8[k] + omega * feq[8] + Flattice[8];
65
66      }
67
68  }
```

Figure 7: (collision_step.cpp) Collision with force function

3 Streaming Step

After the collision step, we have the streaming step. At the streaming step, we look at the populations on each site (lattice node) and move them to the corresponding site after one time step using their velocity $\mathbf{c}_i$.

Using equation (3.10) from the LBM book by Kruger et al,

$$f_i^\star(\mathbf{x}, t) = f_i(\mathbf{x} + \mathbf{c}_i \Delta t, t + \Delta t) \tag{3.10}$$

we could find the new population $f_i^\star(\mathbf{x}, t)$ after streaming.

However, we will use a slightly different approach. Instead of streaming the node populations to the next site, we will work backwards and find which node populations would arrive at a given site.

At a given site (x, y), the post collision populations (remember we did the collision first) that stream to the site (x, y) are given by

$$f_i^\star(x, y, t + 1) = f_i^{col}(x - c_x, y - c_y, t)$$

where $f_i^{col}(x, y, t)$ is the post collision population.

Notice that this requires us to look backwards along the direction $\mathbf{c}_i$ from the site (x, y), yet this is the same as looking forwards along the opposite velocity $-\mathbf{c}_i$. That is we can write,

$$f_i^\star(x, y, t + 1) = f_i^{col}(x + c_x^{opp}, y + c_y^{opp}, t) \tag{S1}$$

where $c_x^{opp} = -c_x$ and $c_y^{opp} = -c_y$. This is known as in-place streaming.

Moreover, the periodic boundary conditions that we are enforcing is given by equation (5.14) from the LBM book by Kruger et al

$$f_i^\star(\mathbf{x}, t) = f_i^\star(\mathbf{x} + \mathbf{L}, t) \tag{5.14}$$

where $\mathbf{L}$ is the periodicity direction and length of the flow pattern. This is relevant now since we could stream 'off' the boundary. This is taken care of in the implementation of the streaming using modular arithmetic.

3.1 Implementation of Streaming Step

For this we call the streaming function, which consists of: Converting the 2D lattice coordinates into a 1d array, calling the function Lattice neighbours, and preforming the streaming step calculation.

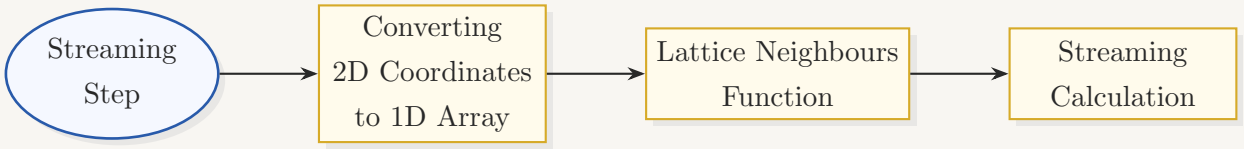


Figure 8: Flow of the Streaming Step implementation.

3.1.1 Converting 2D Coordinates to 1D Array

Currently we have a lattice of size $N = N_x \times N_y$ where N_x and N_y are the lattices in the x/y- direction respectively.

Working from the top left to the bottom right (storing the lattice row by row), the two-dimensional lattice is flattened into a one-dimensional array. Staring at the indexing leads to three conclusions:

- The first row ($y = 0$) takes indices from 0 to $N_x - 1$.
- The index of the first element in row y is $y \times N_x$.
- Moving one lattice site to the right increases the index by one.

Therefore, the lattice site with coordinates (x, y) is stored at the linear index k with

$$k = (y \times N_x) + x$$

```
13  void streaming()
14  {
15
16      for (int y = 0; y < Ny; ++y)
17      {
18
19          for (int x = 0; x < Nx; ++x)
20          {
21
22              // Computation of the linear array index, k, from two-dimensional lattice coordinates (x, y)
23              // The lattices are numbered consecutively along the x-direction and increase in multiples
24              // of the domain length, Nx, in the y-direction.
25              int k = Nx * y + x;
```

Figure 9: (streaming_step.cpp) Defining k

3.1.2 Lattice Neighbours Function

This functions calculates the neighbouring lattices to (x, y) and assigns these neighbours to the array *kneighbour*.

It does so by adding or subtracting one from the current position of (x, y) in the x or y direction modulo N_x or N_y. This is due to the periodic boundary conditions placed on the simulation - a point on the right boundary will be wrapped around to a point on the left boundary.

```cpp
void lattice_neighbours(const int x, const int y, int kneighbour[9])
{

    int xn = (x + 1) % Nx;                      // x-coordinate of neighbouring lattices in the positive x-direction (right)
    int xp = (x - 1 + Nx) % Nx;                 // x-coordinate of neighbouring lattices in the negative x-direction (left)
    int yn = (y + 1) % Ny;                      // y-coordinate of neighbouring lattices in the positive y-direction (up)
    int yp = (y - 1 + Ny) % Ny;                 // y-coordinate of neighbouring lattices in the negative y-direction (down)

    // The coordinates of the neighbouring lattice along the
    // i-th direction are given by: (x + cx[i], y + cy[i]).
    kneighbour[0] = Nx * y + x;
    kneighbour[1] = Nx * y + xn;
    kneighbour[2] = Nx * yn + x;
    kneighbour[3] = Nx * y + xp;
    kneighbour[4] = Nx * yp + x;
    kneighbour[5] = Nx * yn + xn;
    kneighbour[6] = Nx * yn + xp;
    kneighbour[7] = Nx * yp + xp;
    kneighbour[8] = Nx * yp + xn;

}
```

Figure 10: (streaming_step.cpp) Lattice neighbours function

3.1.3 Streaming Calculation

Using *kneighbours*, it preforms the streaming step calculation in (S1) to find the updated distribution after streaming $f_i^\star(\mathbf{x}, t)$.

```cpp

            // "In-place" streaming
            // The post-collision distributions, fcol_i, are streamed from the neighbouring lattices to
            // the k-th lattice, that is:
            // f_i(x, y, t + 1) = fcol_i(x - cx[i], y - cy[i], t) = fcol_i(x + cx[opp[i]], y + cy[opp[i]], t).
            // Periodic boundary conditions, Eq. (5.16) [The Lattice Boltzmann Method: Principles and Practice],
            // are automatically implemented during streaming.
            f1[k] = fcol1[kneighbour[opp[1]]];
            f2[k] = fcol2[kneighbour[opp[2]]];
            f3[k] = fcol3[kneighbour[opp[3]]];
            f4[k] = fcol4[kneighbour[opp[4]]];
            f5[k] = fcol5[kneighbour[opp[5]]];
            f6[k] = fcol6[kneighbour[opp[6]]];
            f7[k] = fcol7[kneighbour[opp[7]]];
            f8[k] = fcol8[kneighbour[opp[8]]];
```

Figure 11: (streaming_step.cpp) Streaming calculation

4 Boundary Conditions

Now that we have preformed the collision and streaming steps, we are ready to apply our boundary conditions.

We have already seen one condition in the streaming step, that is the periodic boundary conditions in (5.14).

Our approach is to utilise four boundary conditions:

(NEBBI) Non-equilibrium bounce back method for the inlet velocity.

(HWBBB) Half-way bounce back method for the straight bottom channel wall.

(HWBBT) Half-way bounce back method for the straight top channel wall.

(NEBBO) Non-equilibrium bounce back method for the outlet pressure.

4.1 NEBBI

The NEBBI condition consists of using the conservation equations

$$\rho = \sum_i f_i, \quad \rho\mathbf{u} = \sum_i f_i \mathbf{c}_i \tag{3.1}$$

which is equation (3.1) in the LBM book by Kruger et al, using the non-equilibrium bounce back condition,

$$f_{\bar{i}}^{\text{neq}}(\mathbf{x}_b, t) = f_i^{\text{neq}}(\mathbf{x}_b, t) \tag{5.42}$$

with $f_i^{\text{neq}} = f_i - f_i^{\text{eq}}$, $\mathbf{c}_{\bar{i}} = -\mathbf{c}_i$ and $\mathbf{x}_b$ is on the boundary; this is equation (5.42) in the LBM book by Kruger et al, and a Poiseuille velocity profile

$$u_x^{in} = U_{max}\left[1 - \left(\frac{2(Y - H/2)}{H}\right)^2\right] \tag{BC1}$$

with U_{max} the maximum velocity, H the channel height, and Y the half-way height, to find the populations f_i.

Since we have a Poiseuille velocity profile at the inlet, $u_y^{in} = 0$ and so $u_y = 0$ and $u_x = u_x^{in}$ (this is due to imposing that $u_x = u_x^{in}$ and $u_y = u_y^{in}$ at the inlet). Using this and (3.1), we arrive at an expression for the mass density at the inlet

$$\rho = \frac{f_0 + f_2 + f_4 + 2(f_3 + f_6 + f_7)}{1 - u_x^{in}} \tag{BC2}$$

in terms of u_x^{in} and the populations $f_0, f_2, f_3, f_4, f_6, f_7$,. From the streaming step, we know these values. However, we do not know the populations f_1, f_5, f_8 since these would have required fluid streaming in from $x = -1$ (in lattice units) which is outside of our channel.

Applying the non-equilibrium bounce back condition (5.42), to $i = 1$, $\bar{i} = 3$ results in

$$f_1 - f_1^{\text{eq}} = f_3 - f_3^{\text{eq}}$$

which coupled with the conservation equations (3.1), gives a system which we can solve for the remaining unknown populations

$$\begin{aligned} f_1 &= f_3 + \frac{2}{3}\rho u_x^{in} \\ f_5 &= f_7 - \frac{1}{2}(f_2 - f_4) + \frac{1}{6}\rho u_x^{in} \\ f_8 &= f_6 + \frac{1}{2}(f_2 - f_4) + \frac{1}{6}\rho u_x^{in} \end{aligned} \tag{BC3}$$

The two corners at the top and bottom walls require special treatment, as we lose even more information about the populations than at an interior inlet node, resulting in the populations f_1, f_2, f_5, f_6, f_8 being unknown at the bottom corner, and f_1, f_4, f_5, f_7, f_8 being unknown at the top corner.

Firstly, the Poiseuille velocity profile is calculated at the half-way height corresponding to each corner, using the same form as at the boundary,

$$u_x^{in} = U_{max}\left[1 - \left(\frac{2(Y - H/2)}{H}\right)^2\right] \tag{BC4}$$

with $Y = 0.5$ at the bottom corner and $Y = (y_{max} - y_{min}) + 0.5$ at the top corner.

Secondly, the mass density ρ is not calculated from the site population at the corner; instead, it is borrowed from the neighbouring node one lattice spacing into the fluid — directly above for the bottom corner, and directly below for the top corner.

For the bottom corner, the specific bounce-back relation is applied to give

$$f_2 = f_4, \qquad f_8 = -f_6$$

which we can couple with the conservation equations (3.1) and the non-equilibrium bounce-back condition (5.42) applied to f_1, f_3, to find a system which we can solve for the remaining unknown populations

$$\begin{aligned} f_1 &= f_3 + \frac{2}{3}\rho u_x^{in} \\ f_5 &= f_7 + \frac{1}{6}\rho u_x^{in} \\ f_6 &= -\frac{1}{12}\rho u_x^{in} \\ f_0 &= \rho - \sum_{i=1}^{8} f_i \end{aligned} \tag{BC5}$$

Similarly for the top corner, the specific bounce-back relation

$$f_4 = f_2, \qquad f_7 = -f_5$$

is used in conjunction with (3.1) and (5.42) applied to f_1, f_3, giving

$$\begin{aligned} f_1 &= f_3 + \frac{2}{3}\rho u_x^{in} \\ f_5 &= \frac{1}{12}\rho u_x^{in} \\ f_8 &= f_6 + \frac{1}{6}\rho u_x^{in} \\ f_0 &= \rho - \sum_{i=1}^{8} f_i \end{aligned} \tag{BC6}$$

It is worth noting that f_0 is not unknown after streaming, and it is recomputed here to force exact mass conservation at the corner.

4.2 HWBBB

The HWBBB consist of using the half-way bounce back condition

$$f_{\bar{i}}(\mathbf{x}_b, t + \Delta t) = f_i^{\star}(\mathbf{x}_b, t) \tag{5.24}$$

which is equation (5.24) from the LBM book by Kruger et al. This equation applied to the populations f_2, f_5, f_6 results in

$$\begin{aligned} f_2 &= f_4^{\star} \\ f_5 &= f_7^{\star} \\ f_6 &= f_8^{\star} \end{aligned} \tag{BC7}$$

4.3 HWBBT

Similarly to the HWBBB, the HWBBT consists of using the half-way bounce back condition (5.24) when applied to f_4, f_7, f_8,

$$\begin{aligned} f_4 &= f_2^{\star} \\ f_7 &= f_5^{\star} \\ f_8 &= f_6^{\star} \end{aligned} \tag{BC8}$$

which coincidentally (it is not a coincidence) is the same as (BC7) just written in the reverse order.

4.4 NEBBO

A uniform fluid density profile is considered here at the outlet, given by:

$$\rho_{out} = \rho_{ref} - \frac{\Delta\rho}{2}$$

where $\rho_{ref} = 1.0$ is the fluid density reference value, and $\Delta\rho = \Delta p/c_s^2$ is the fluid density difference between the inlet and outlet of the domain.

The fluid pressure difference, Δp, can be found by the planar Poiseuille flow as:

$$\Delta p = \frac{8\nu\rho_{ref} L U_{max}}{H^2}$$

where L and H denote the length and height of the channel, respectively. Using this, an expression for the mass density at the outlet ρ_{out} is found as

$$\rho_{out} = 1 - \frac{4\nu(N_x - 1)U_{max}}{c_s^2 H^2} \tag{BC9}$$

Using the conservation equations (3.1), we can solve for the outlet velocity in the x-direction u_x^{out} as

$$u_x^{out} = \frac{f_0 + f_2 + f_4 + 2(f_1 + f_5 + f_8)}{\rho_{out}} - 1 \tag{BC10}$$

Conversely, we borrow u_y^{out} from the left neighbour as

$$u_y^{out} = \frac{(f_2 + f_5 + f_6) - (f_4 + f_7 + f_8)}{\rho_{left}} \tag{BC11}$$

where $\rho_{left} = \sum_i f_i(x-1, y, t)$.

Finally, using the non-equilibrium bounce back conditions (5.42) to f_1 and f_3, namely

$$f_1 - f_1^{\text{eq}} = f_3 - f_3^{\text{eq}}$$

we can find expressions for f_3, f_6, f_7 as

$$\begin{aligned} f_3 &= f_1 - \frac{2}{3}\rho_{out} u_x^{out} \\ f_6 &= f_8 - \frac{1}{2}(f_2 - f_4) - \frac{1}{6}\rho_{out} u_x^{out} + \frac{1}{2}\rho_{out} u_y^{out} \\ f_7 &= f_5 + \frac{1}{2}(f_2 - f_4) - \frac{1}{6}\rho_{out} u_x^{out} - \frac{1}{2}\rho_{out} u_y^{out} \end{aligned} \tag{BC12}$$

The two corners at the top and bottom walls require special treatment, as we lose even more information about the populations than at an interior outlet node, resulting in the populations f_2, f_3, f_5, f_6, f_7 being unknown at the bottom corner, and f_3, f_4, f_6, f_7, f_8 being unknown at the top corner.

Firstly, since neither u_x^{out} or u_y^{out} can be found locally from the conservation equations (3.1) with only 4 known populations, both are instead borrowed from the neighbouring node one lattice spacing to the left,

$$u_x^{out} = \frac{(f_1 + f_5 + f_8) - (f_3 + f_6 + f_7)}{\rho_{left}}, \qquad u_y^{out} = \frac{(f_2 + f_5 + f_6) - (f_4 + f_7 + f_8)}{\rho_{left}} \tag{BC13}$$

where $\rho_{left} = \sum_i f_i(x-1, y, t)$, evaluated at the appropriate corner row.

For the bottom corner, the specific bounce-back relation is applied to give

$$f_7 = -f_5$$

which we use in conjunction with the conservation equations (3.1) and the non-equilibrium bounce-back condition (5.42) applied to f_1, f_3, to find a system which we can solve for the remaining unknown populations

$$\begin{aligned} f_2 &= f_4 + \frac{2}{3}\rho_{out}u_y^{out} \\ f_3 &= f_1 - \frac{2}{3}\rho_{out}u_x^{out} \\ f_5 &= \frac{1}{12}\rho_{out}u_x^{out} + \frac{1}{12}\rho_{out}u_y^{out} \\ f_6 &= f_8 - \frac{1}{6}\rho_{out}u_x^{out} + \frac{1}{6}\rho_{out}u_y^{out} \\ f_0 &= \rho_{out} - \sum_{i=1}^{8} f_i \end{aligned} \tag{BC14}$$

Similarly for the top corner, the specific bounce-back relation

$$f_8 = -f_6$$

is used in conjunction with (3.1) and (5.42) applied to f_1, f_3, giving

$$\begin{aligned} f_3 &= f_1 - \frac{2}{3}\rho_{out}u_x^{out} \\ f_4 &= f_2 - \frac{2}{3}\rho_{out}u_y^{out} \\ f_6 &= -\frac{1}{12}\rho_{out}u_x^{out} + \frac{1}{12}\rho_{out}u_y^{out} \\ f_7 &= f_5 - \frac{1}{6}\rho_{out}u_x^{out} - \frac{1}{6}\rho_{out}u_y^{out} \\ f_0 &= \rho_{out} - \sum_{i=1}^{8} f_i \end{aligned} \tag{BC15}$$

As with the NEBBI corners, f_0 is not really unknown after streaming and is recomputed here to force exact mass conservation at the corner.

4.5 Implementation of Boundary Conditions

For this we call the four functions: the NEBB inlet velocity function (NEBBI), HWBB bottom wall function (HWBBB), HWBB top wall function (HWBBT) and the NEBB outlet presure function (NEBBO).

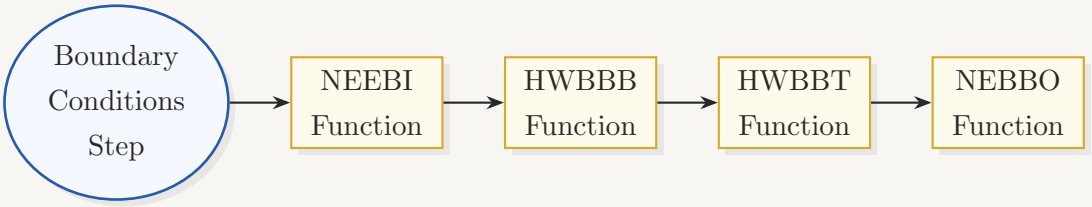


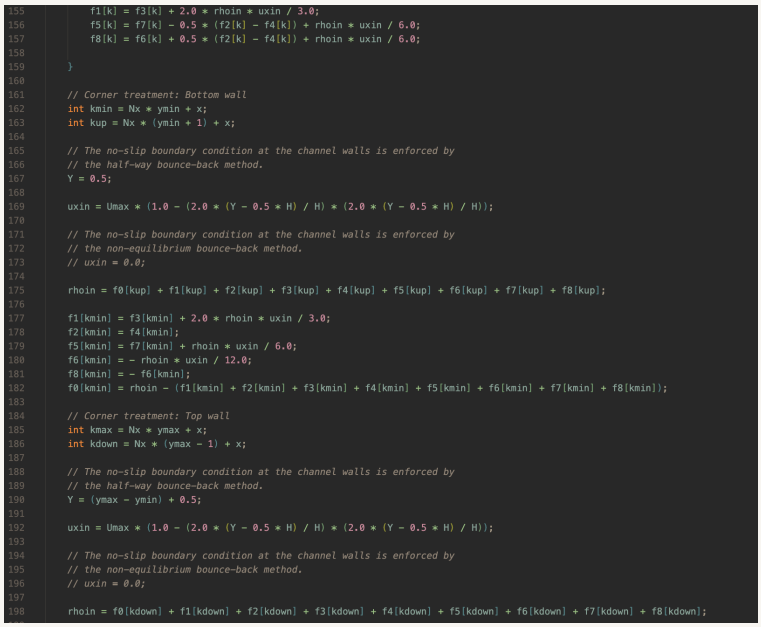
Figure 12: Flow of the Boundary Conditions Step implementation.

4.5.1 NEBBI Function

The function uses the results found in (BC1), (BC2), (BC3), (BC4), (BC5) and (BC6).

```
110  void NEBB_inlet_velocity(int ymin, int ymax, int x)
111  {
112
113      double Y; // y-coordinate (physical domain)
114      double uxin; // x-component of the fluid velocity at the inlet
115      double rhoin; // fluid density at the inlet
116
117      // A Poiseuille flow profile is considered here along the y-direction.
118
119      // The no-slip boundary condition at the channel walls is enforced by
120      // the half-way bounce-back method.
121      // In the channel's height (H) computation, 1 is added to account for
122      // the distance between (x, ymin) / (x, ymax) and the channel wall -
123      // the channel wall resides at a distance of 0.5 lattice spacing from
124      // the aforementioned lattices.
125      double H = ymax - ymin + 1.0;
126
127      // The no-slip boundary condition at the channel walls is enforced by
128      // the non-equilibrim bounce-back method.
129      // double H = ymax - ymin;
130
131      // Intermediate lattices treatment
132      for (int y = (ymin + 1); y < ymax; ++y)
133      {
134
135          int k = Nx * y + x;
136
137          // The no-slip boundary condition at the channel walls is enforced by
138          // the half-way bounce-back method.
139          Y = (y - ymin) + 0.5;
140
141          // The no-slip boundary condition at the channel walls is enforced by
142          // the non-equilibrim bounce-back method.
143          // Y = y;
144
145          // In the Poiseuille flow profile, only the x-component of the fluid velocity exists,
146          // with the y-component (uyin) being 0.
147          // For this reason, the terms involving uyin have been neglected in the following!
148          uxin = Umax * (1.0 - (2.0 * (Y - 0.5 * H) / H) * (2.0 * (Y - 0.5 * H) / H));
149
150          // Remember: If uyin is not equal to 0, then the terms involving it should be added in the following!
151          // Remember: If a (total) Eulerian force density field exists, then the terms involving it should be
152          //           added in the following!
153          rhoin = (f0[k] + f2[k] + f4[k] + 2.0 * (f3[k] + f6[k] + f7[k])) / (1.0 - uxin);
```

Figure 13: (boundary_conditions.cpp) NEBBI function ss1



```
        f1[k] = f3[k] + 2.0 * rhoin * uxin / 3.0;
        f5[k] = f7[k] - 0.5 * (f2[k] - f4[k]) + rhoin * uxin / 6.0;
        f8[k] = f6[k] + 0.5 * (f2[k] - f4[k]) + rhoin * uxin / 6.0;

    }

    // Corner treatment: Bottom wall
    int kmin = Nx * ymin + x;
    int kup = Nx * (ymin + 1) + x;

    // The no-slip boundary condition at the channel walls is enforced by
    // the half-way bounce-back method.
    Y = 0.5;

    uxin = Umax * (1.0 - (2.0 * (Y - 0.5 * H) / H) * (2.0 * (Y - 0.5 * H) / H));

    // The no-slip boundary condition at the channel walls is enforced by
    // the non-equilibrium bounce-back method.
    // uxin = 0.0;

    rhoin = f0[kup] + f1[kup] + f2[kup] + f3[kup] + f4[kup] + f5[kup] + f6[kup] + f7[kup] + f8[kup];

    f1[kmin] = f3[kmin] + 2.0 * rhoin * uxin / 3.0;
    f2[kmin] = f4[kmin];
    f5[kmin] = f7[kmin] + rhoin * uxin / 6.0;
    f6[kmin] = - rhoin * uxin / 12.0;
    f8[kmin] = - f6[kmin];
    f0[kmin] = rhoin - (f1[kmin] + f2[kmin] + f3[kmin] + f4[kmin] + f5[kmin] + f6[kmin] + f7[kmin] + f8[kmin]);

    // Corner treatment: Top wall
    int kmax = Nx * ymax + x;
    int kdown = Nx * (ymax - 1) + x;

    // The no-slip boundary condition at the channel walls is enforced by
    // the half-way bounce-back method.
    Y = (ymax - ymin) + 0.5;

    uxin = Umax * (1.0 - (2.0 * (Y - 0.5 * H) / H) * (2.0 * (Y - 0.5 * H) / H));

    // The no-slip boundary condition at the channel walls is enforced by
    // the non-equilibrium bounce-back method.
    // uxin = 0.0;

    rhoin = f0[kdown] + f1[kdown] + f2[kdown] + f3[kdown] + f4[kdown] + f5[kdown] + f6[kdown] + f7[kdown] + f8[kdown];
```

Figure 14: (boundary_conditions.cpp) NEBBI function ss2

```
    f1[kmax] = f3[kmax] + 2.0 * rhoin * uxin / 3.0;
    f4[kmax] = f2[kmax];
    f5[kmax] = rhoin * uxin / 12.0;
    f7[kmax] = - f5[kmax];
    f8[kmax] = f6[kmax] + rhoin * uxin / 6.0;
    f0[kmax] = rhoin - (f1[kmax] + f2[kmax] + f3[kmax] + f4[kmax] + f5[kmax] + f6[kmax] + f7[kmax] + f8[kmax]);

}
```

Figure 15: (boundary_conditions.cpp) NEBBI function ss3

4.5.2 HWBBB Function

This function preforms the calculations from (BC7).

```
// Bottom wall
// The wall extends between xmin and xmax (in the x-direction) and it is located at height y.
void HWBB_bottom_wall(int xmin, int xmax, int y)
{

    // Computation of the linear array indices kmin and kmax from the two-dimensional
    // lattice coordinates (xmin, y) and (xmax, y), respectively.
    // In the kmax computation, 1 is added to ensure that the boundary condition is
    // applied at the wall's distal end, i.e. at (xmax, y).
    int kmin = Nx * y + xmin;
    int kmax = Nx * y + xmax + 1;

    for (int k = kmin; k < kmax; ++k)
    {

        f2[k] = fcol4[k];
        f5[k] = fcol7[k];
        f6[k] = fcol8[k];

    }

}
```

Figure 16: (boundary_conditions.cpp) HWBBB function

4.5.3 HWBBT Function

This function preforms the calculations from (BC8).

```
// Top wall
// The wall extends between xmin and xmax (in the x-direction) and it is located at height y.
void HWBB_top_wall(int xmin, int xmax, int y)
{

    int kmin = Nx * y + xmin;
    int kmax = Nx * y + xmax + 1;

    for (int k = kmin; k < kmax; ++k)
    {

        f4[k] = fcol2[k];
        f7[k] = fcol5[k];
        f8[k] = fcol6[k];

    }

}
```

Figure 17: (boundary_conditions.cpp) HWBBT function

4.5.4 NEBBO Function

This function preforms the calculations in (BC9), (BC10), (BC11), (BC12), (BC13), (BC14) and (BC15).

```
void NEBB_outlet_pressure(int ymin, int ymax, int x)
{

    double uxout; // x-component of the fluid velocity at the outlet
    double uyout; // y-component of the fluid velocity at the outlet

    // A uniform fluid density profile is considered here at the outlet, given by:
    // rhoout = rho_ref - Drho / 2,
    // where rho_ref = 1.0 is the fluid density reference value, and Drho = Dp / cs^2
    // is the fluid density difference between the inlet and outlet of the domain.
    // The fluid pressure difference, Dp, can be found by the planar Poiseuille flow as:
    // Dp = 8 * nu * rho_ref * L * Umax / H^2, where L and H denote the length and height
    // of the channel, respectively.

    // The no-slip boundary condition at the channel walls is enforced by
    // the half-way bounce-back method.
    double H = ymax - ymin + 1.0;

    // The no-slip boundary condition at the channel walls is enforced by
    // the non-equilibrim bounce-back method.
    // double H = ymax - ymin;

    // Remember: It is assumed that the channel's length is equal to the length of the
    //           computational domain.
    double rhoout = 1.0 - 4.0 * nu * (Nx - 1.0) * Umax / (cssq * H * H);

    // Intermediate lattices treatment
    for (int y = (ymin + 1); y < ymax; ++y)
    {

        int k = Nx * y + x;
        int kleft = Nx * y + (x - 1);

        // Remember: If a (total) Eulerian force density field exists, then the terms involving it should be
        //           added in the following!
        double rholeft = f0[kleft] + f1[kleft] + f2[kleft] + f3[kleft] + f4[kleft] + f5[kleft] + f6[kleft] + f7[kleft] + f8[kleft];

        uxout = (f0[k] + f2[k] + f4[k] + 2.0 * (f1[k] + f5[k] + f8[k])) / rhoout - 1.0;
        uyout = (f2[kleft] + f5[kleft] + f6[kleft] - (f4[kleft] + f7[kleft] + f8[kleft])) / rholeft;

        f3[k] = f1[k] - 2.0 * rhoout * uxout / 3.0;
        f6[k] = f8[k] - 0.5 * (f2[k] - f4[k]) - rhoout * uxout / 6.0 + 0.5 * rhoout * uyout;
        f7[k] = f5[k] + 0.5 * (f2[k] - f4[k]) - rhoout * uxout / 6.0 - 0.5 * rhoout * uyout;

    }

    // Corner treatment: Bottom wall
    int kmin = Nx * ymin + x;
    int kleftb = Nx * ymin + (x - 1);
```

Figure 18: (boundary_conditions.cpp) NEBBO function ss1

```
    // The no-slip boundary condition at the channel walls is enforced by
    // the half-way bounce-back method.
    double rholeftb = f0[kleftb] + f1[kleftb] + f2[kleftb] + f3[kleftb] + f4[kleftb] + f5[kleftb] + f6[kleftb] + f7[kleftb] + f8[kleftb];

    uxout = ((f1[kleftb] + f5[kleftb] + f8[kleftb]) - (f3[kleftb] + f6[kleftb] + f7[kleftb])) / rholeftb;
    uyout = (f2[kleftb] + f5[kleftb] + f6[kleftb] - (f4[kleftb] + f7[kleftb] + f8[kleftb])) / rholeftb;

    // The no-slip boundary condition at the channel walls is enforced by
    // the non-equilibrium bounce-back method.
    // uxout = 0.0;
    // uyout = 0.0;

    f2[kmin] = f4[kmin] + 2.0 * rhoout * uyout / 3.0;
    f3[kmin] = f1[kmin] - 2.0 * rhoout * uxout / 3.0;
    f5[kmin] = rhoout * uxout / 12.0 + rhoout * uyout / 12.0;
    f6[kmin] = f8[kmin] - rhoout * uxout / 6.0 + rhoout * uyout / 6.0;
    f7[kmin] = - f5[kmin];
    f0[kmin] = rhoout - (f1[kmin] + f2[kmin] + f3[kmin] + f4[kmin] + f5[kmin] + f6[kmin] + f7[kmin] + f8[kmin]);

    // Corner treatment: Top wall
    int kmax = Nx * ymax + x;
    int kleftt = Nx * ymax + (x - 1);

    // The no-slip boundary condition at the channel walls is enforced by
    // the half-way bounce-back method.
    double rholeftt = f0[kleftt] + f1[kleftt] + f2[kleftt] + f3[kleftt] + f4[kleftt] + f5[kleftt] + f6[kleftt] + f7[kleftt] + f8[kleftt];

    uxout = ((f1[kleftt] + f5[kleftt] + f8[kleftt]) - (f3[kleftt] + f6[kleftt] + f7[kleftt])) / rholeftt;
    uyout = (f2[kleftt] + f5[kleftt] + f6[kleftt] - (f4[kleftt] + f7[kleftt] + f8[kleftt])) / rholeftt;

    // The no-slip boundary condition at the channel walls is enforced by
    // the non-equilibrium bounce-back method.
    // uxout = 0.0;
    // uyout = 0.0;

    f3[kmax] = f1[kmax] - 2.0 * rhoout * uxout / 3.0;
    f4[kmax] = f2[kmax] - 2.0 * rhoout * uyout / 3.0;
    f6[kmax] = - rhoout * uxout / 12.0 + rhoout * uyout / 12.0;
    f7[kmax] = f5[kmax] - rhoout * uxout / 6.0 - rhoout * uyout / 6.0;
    f8[kmax] = - f6[kmax];
    f0[kmax] = rhoout - (f1[kmax] + f2[kmax] + f3[kmax] + f4[kmax] + f5[kmax] + f6[kmax] + f7[kmax] + f8[kmax]);

}
```

Figure 19: (boundary_conditions.cpp) NEBBO function ss2

5 Macroscopic Moments

Now that the collision step, streaming step and boundary conditions are applied, the program calculates the macroscopic moments.

Similar to the collision steps, there are two types of macroscopic moments: moments with a force or without a force. Currently implemented is macroscopic moments without a force.

5.1 Macroscopic Moments Without a Force

For macroscopic moments without a force, we use the conservation equations in (3.1) to find the mass density ρ and velocity $\mathbf{u}$. Using the velocity set in Table 1, we find expressions

$$\rho = f_0 + f_1 + f_2 + f_3 + f_4 + f_5 + f_6 + f_7 + f_8 \tag{M1}$$

$$u_x = \frac{1}{\rho}\left[(f_1 + f_5 + f_8) - (f_3 + f_6 + f_7)\right] \tag{M2}$$

$$u_y = \frac{1}{\rho}\left[(f_2 + f_5 + f_6) - (f_4 + f_7 + f_8)\right] \tag{M3}$$

5.2 Implementation of Macroscopic Moments Without a Force

Currently in the main file of the program, we have the macroscopic moments function which preforms the calculations outlined in (M1), (M2) and (M3).

```cpp
void macroscopic_moments()
{

    for (int k = 0; k < N; ++k)
    {

        rho[k] = f0[k] + f1[k] + f2[k] + f3[k] + f4[k] + f5[k] + f6[k] + f7[k] + f8[k];

        double rhoinv = 1.0 / rho[k];

        ux[k] = rhoinv * ((f1[k] + f5[k] + f8[k]) - (f3[k] + f6[k] + f7[k]));
        uy[k] = rhoinv * ((f2[k] + f5[k] + f6[k]) - (f4[k] + f7[k] + f8[k]));

    }

}
```

Figure 20: (macroscopicmoments.cpp) macroscopic moments function

5.3 Macroscopic Moments With a Force

For macroscopic moments with a force, we use the same mass conservation equation in (3.1) but a different momentum conservation equation due to the addition of a force $\mathbf{F}$,

$$\rho \mathbf{u} = \sum_i f_i \mathbf{c}_i + \frac{\Delta t}{2} \mathbf{F} \tag{17}$$

Using the velocity set in Table 1 and lattice units, we find expressions

$$\rho = f_0 + f_1 + f_2 + f_3 + f_4 + f_5 + f_6 + f_7 + f_8 \tag{M4}$$

$$u_x = \frac{1}{\rho}\left[(f_1 + f_5 + f_8) - (f_3 + f_6 + f_7) + \frac{1}{2}F_x\right] \tag{M5}$$

$$u_y = \frac{1}{\rho}\left[(f_2 + f_5 + f_6) - (f_4 + f_7 + f_8) + \frac{1}{2}F_y\right] \tag{M6}$$

5.4 Implementation of Macroscopic Moments With a Force

For this, the macroscopic moments with force function which preforms the calculations outlined in (M4), (M5) and (M6).

```cpp
void macroscopic_moments_with_force()
{

    for (int k = 0; k < N; ++k)
    {

        rho[k] = f0[k] + f1[k] + f2[k] + f3[k] + f4[k] + f5[k] + f6[k] + f7[k] + f8[k];

        double rhoinv = 1.0 / rho[k];

        ux[k] = rhoinv * ((f1[k] + f5[k] + f8[k]) - (f3[k] + f6[k] + f7[k]) + 0.5 * fx[k]);
        uy[k] = rhoinv * ((f2[k] + f5[k] + f6[k]) - (f4[k] + f7[k] + f8[k]) + 0.5 * fy[k]);

    }

}
```

Figure 21: (macroscopicmoments.cpp) macroscopic moments with force function

6 L_2-norm Error Calculation

Once the main steps (initialisation, collision, streaming, boundary conditions and moments) have been completed, the program calculates the L_2-norm error of the velocity field $\mathbf{u}(\mathbf{x}, t)$.

Given a known analytic quantity $q_a(\mathbf{x}, t)$ and its numerical equivalent $q_n(\mathbf{x}, t)$, the L_2-norm error of $q_n(\mathbf{x}, t)$ is defined as

$$\epsilon_q(t) = \sqrt{\frac{\sum_{\mathbf{x}}[q_n(\mathbf{x}, t) - q_a(\mathbf{x}, t)]^2}{\sum_{\mathbf{x}} q_a^2(\mathbf{x}, t)}} \tag{4.57}$$

where the sum runs over the entire spacial domain in which $q_a(\mathbf{x}, t)$ is defined; this is equation (4.57) in the LBM book by Kruger et al.

Taking $q_a(\mathbf{x}, t)$ and $q_n(\mathbf{x}, t)$ to be the vector quantities $\mathbf{u}(\mathbf{x}, t)$ and $\mathbf{u}^{old}(\mathbf{x}, t)$, (4.57) reads,

$$\epsilon_q(t) = \sqrt{\frac{\sum_{\mathbf{x}}[\mathbf{u}(\mathbf{x}, t) - \mathbf{u}^{old}(\mathbf{x}, t)]^2}{\sum_{\mathbf{x}} |\mathbf{u}^{old}(\mathbf{x}, t)|^2}} \tag{E1}$$

where $\mathbf{u}^{old}(\mathbf{x}, t)$ will be the previous value of the velocity field in the iteration step.

Once the error is calculated, the program checks if this value is less than 10^{-13}; if this is true it breaks the for loop.

6.1 Implementation of L_2-norm Error Calculation

For this, a for loop over all space is written in with the calculation in (E1) in components.

```
130         // L2-norm error of the fluid velocity field between successive iterations
131         num = 0.0;
132         den = 0.0;
133         for (int k = 0; k < N; ++k)
134         {
135 
136             num = num + (ux[k] - uxold[k]) * (ux[k] - uxold[k]) + (uy[k] - uyold[k]) * (uy[k] - uyold[k]);
137             den = den + uxold[k] * uxold[k] + uyold[k] * uyold[k];
138 
139             uxold[k] = ux[k];
140             uyold[k] = uy[k];
141 
142         }
143 
144         error = sqrt(num/den);
145 
146     }
```

Figure 22: (main.cpp) L_2-norm error calculation

At the start of the main for loop, an if statement checks the size of the error.

```
63          // Save the desired output data at the end of the simulation, when the main
64          // loop breaks due to the satisfaction of the convergence criterion
65          if (error < 1e-12)
66          {
67
68              save_output(t);
69
70              break;
71
72          }
```

Figure 23: (main.cpp) L_2-norm error size check